

Penggunaan Struktur Data Stack dalam Aplikasi Reservasi Barbershop (NeBar)

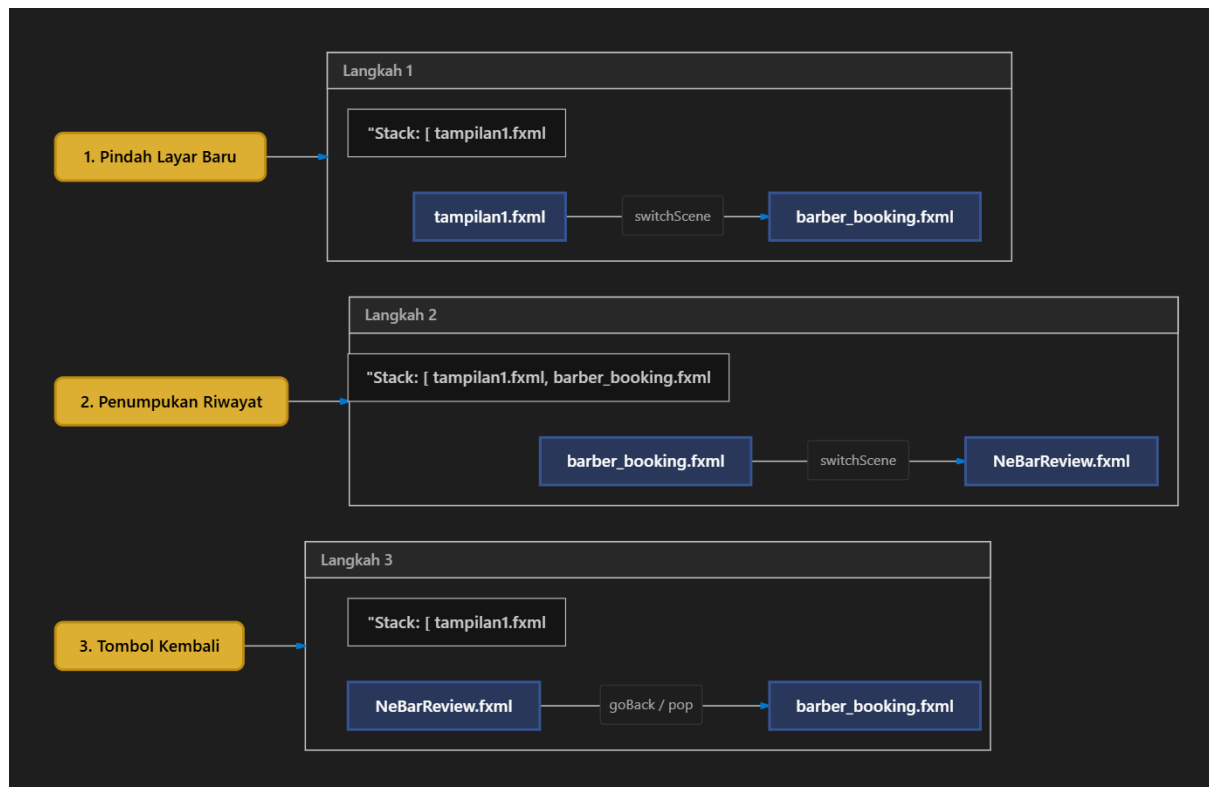
Aplikasi Reservasi Barbershop (NeBar) yang dikembangkan pada proyek ini menggunakan struktur data Stack (Tumpukan) untuk mengelola riwayat navigasi antar-halaman (navigation history). Pemilihan Stack dilakukan karena alur perpindahan halaman pada aplikasi menuntut prinsip LIFO (Last In First Out), yaitu halaman terakhir yang dibuka oleh pengguna harus menjadi halaman pertama yang muncul kembali ketika tombol kembali ("Back") ditekan. Dalam implementasinya, riwayat navigasi disimpan pada variabel history yang bertipe Stack<String> di dalam kelas utama main.App. Dengan demikian, urutan kembali halaman tidak perlu di-hardcode secara manual di setiap tombol, melainkan diatur secara dinamis oleh tumpukan yang berjalan di memori.

Kelas Stack<E> yang digunakan pada proyek ini memanfaatkan pustaka bawaan Java (java.util.Stack). Operasi dasar navigasi memanfaatkan metode utama dari struktur data Stack, terutama push() untuk memasukkan nama file FXML halaman saat ini ke puncak tumpukan sebelum berpindah layar, dan pop() untuk mengambil sekaligus menghapus halaman teratas dari tumpukan saat pengguna ingin kembali ke halaman sebelumnya. Pendekatan ini membuat manajemen navigasi aplikasi menjadi lebih rapi, konsisten, dan meminimalisir kesalahan rute halaman.

1. Keterkaitan Struktur Data dengan Alur pada Gambar 1

(Catatan: Gambar 1 di sini diasumsikan sebagai diagram alur perpindahan halaman: Layar Login → Layar Dashboard Utama → Layar Pemesanan Barber → Tombol Kembali → Layar Dashboard Utama).

Gambar 1 menunjukkan alur kerja sistem navigasi halaman yang diterapkan pada aplikasi:



Pada nomor 1 (Pindah Layar Baru): Pengguna berada di layar dashboard (tampilan1.fxml) dan memilih untuk masuk ke layar pemesanan (barber_booking.fxml). Sebelum layar berpindah, metode switchScene() akan dipanggil dan sistem melakukan operasi push() untuk memasukkan "tampilan1.fxml" ke dalam tumpukan history.

Pada nomor 2 (Penumpukan Riwayat): Jika pengguna melanjutkan navigasi ke halaman ulasan (NeBarReview.fxml), maka halaman pemesanan sebelumnya akan ditumpuk lagi di atas tumpukan riwayat, sehingga posisi riwayat saat ini adalah [tampilan1.fxml, barber_booking.fxml] dengan halaman pemesanan berada di puncak (top).

Pada nomor 3 (Eksekusi Tombol Kembali): Ketika pengguna menekan tombol "Back", sistem memanggil metode goBack(). Langkah pertama adalah mengecek tumpukan dengan isEmpty(), lalu mengambil data halaman teratas dengan operasi pop() untuk memuat kembali halaman "barber_booking.fxml". Halaman tersebut otomatis terhapus dari tumpukan sehingga tumpukan riwayat kembali menyusut.

2. Peran Variabel history dalam Pengelolaan Navigasi

Pada kelas utama main.App.java, variabel history berfungsi sebagai struktur data kerja yang menentukan jalannya alur balik halaman secara mutlak:

history.push(currentScene): Berfungsi untuk merekam jejak halaman yang sedang aktif tepat sebelum pengguna dialihkan ke layar baru.

history.pop(): Berfungsi untuk menarik mundur satu langkah halaman terakhir yang dikunjungi pengguna dari tumpukan untuk segera ditampilkan ulang di layar utama.

history.isEmpty(): Digunakan sebagai gerbang validasi di metode goBack(). Langkah pengecekan ini sangat penting agar aplikasi tidak mengalami error (EmptyStackException)

apabila pengguna mencoba menekan tombol kembali saat tumpukan riwayat navigasi sudah kosong.

3. Evaluasi Pengembangan

Selain fungsi dasar tumpukan navigasi, ada hal yang dapat dipandang sebagai evaluasi pengembangan lanjutan pada manajemen Stack ini. Pada aplikasi saat ini, tumpukan riwayat akan terus bertambah panjang tanpa batas selama pengguna terus berpindah-pindah halaman secara berputar (looping navigasi). Hal ini berpotensi menyebabkan penggunaan memori yang tidak efisien jika pengguna menggunakan aplikasi dalam waktu yang sangat lama.

Sebagai pengembangan lanjutan, sistem navigasi dapat ditingkatkan dengan membatasi kapasitas maksimal Stack (Bounded Stack) atau dengan melakukan pembersihan (clear history) secara otomatis saat terjadi aksi penting tertentu, seperti ketika pengguna menekan tombol Logout atau setelah proses pemesanan dinyatakan berhasil (Done). Penyesuaian ini akan menjaga konsistensi alur aplikasi sekaligus mengoptimalkan konsumsi memori perangkat.

4. Alasan Stack Sesuai untuk Navigasi Halaman

Struktur data Stack sangat sesuai untuk mengelola riwayat navigasi karena perilakunya secara alami selaras dengan cara kerja tombol "Back" pada aplikasi modern. Pengguna mengharapkan alur mundur yang runtut terbalik dari cara mereka melangkah masuk.

Dengan memanfaatkan sifat LIFO pada Stack, pengembang tidak perlu menulis logika pengkondisian (if-else) yang rumit untuk memetakan tombol kembali di setiap file controller FXML. Cukup dengan satu metode global goBack() yang memanfaatkan operasi pop(), semua tombol kembali di layar manapun akan otomatis mengantarkan pengguna ke halaman yang tepat sebelumnya.

5. Kesimpulan

Berdasarkan implementasi pada proyek ini, Stack merupakan struktur data yang sangat tepat untuk menangani sistem navigasi aplikasi Reservasi Barbershop (NeBar). Struktur data ini memungkinkan pencatatan rute halaman secara berurutan, eksekusi tombol kembali yang dinamis, dan pencegahan kerusakan alur navigasi melalui validasi tumpukan kosong. Penerapan batas tumpukan riwayat di masa mendatang akan semakin meningkatkan stabilitas dan efisiensi memori aplikasi ini.